

Pràctica 5: Prolog avançat

Lògica en la Informàtica

FIB

Antoni Lozano

Q1 2025–2026

Objectius

Aquesta pràctica té com a objectius:

- Resoldre problemes més avançats de caire combinatori.
- Fer ús d'un esquema predeterminat que permet explorar un espai d'estats de mida exponencial.

Disposeu de:

- Un pdf amb l'enunciat de la pràctica dividida en apartats A, B i C.
- El fitxer `camí_cost_minim.pl` que conté un esquema d'exploració de solucions (per utilitzar en els problemes B).
- El fitxer `tsp.pl` on cal resoldre el problema C.

A: Xifres amb divisió

Adapta el problema de “xifres” de les pàgines 14–15 del tema **p6** a la *Col·lecció d'apunts bàsics de lògica* perquè:

- 1 també faci servir la divisió (sense donar error de divisió per zero),
- 2 trobi les solucions més curtes primer (les que usen menys elements de la llista L) i
- 3 comenci donant la solució exacta i, després, les que tinguin distàncies més grans.

A: Xifres amb divisió

```
% xifres(L,N) escriu les maneres d'obtenir N a partir de +, -, *  
% dels elements de la llista L
```

```
% exemple:
```

```
% ?- xifres([4,9,8,7,100,4], 380).
```

```
% ... 4 * (100-7) + 8 ... <-----
```

```
% ... ((100-9) + 4) * 4
```

```
% ... ..
```

```
xifres(L,N):-
```

```
... subconjunt(L,S), ... % S = [4,8,7,100]
```

```
... permutation(S,P), ... % P = [4,100,7,8]
```

```
... expressio(P,E), ... % E = 4 * (100-7) + 8
```

```
... N is E,
```

```
... write(E), nl, false.
```

```
% E = ( 4 * ( 100 - 7 ) ) + 8
```

```
% ..... +
```

```
% ..... / \
```

```
% ..... * ..... 8
```

```
% ..... / \
```

```
% ..... 4 -
```

```
% ..... / \
```

```
% ..... 100 - 7
```

A: Xifres amb divisió

```
expressio([X],X).
```

```
expressio(L, E1 + E2):-
```

→	→	→	· append(L1, L2, L),
→	→	→	· L1 \= [], L2 \= [],
→	→	→	· expressio(L1, E1),
→	→	→	· expressio(L2, E2).

```
expressio(L, E1 - E2):-
```

→	→	→	· append(L1, L2, L),
→	→	→	· L1 \= [], L2 \= [],
→	→	→	· expressio(L1, E1),
→	→	→	· expressio(L2, E2).

```
expressio(L, E1 * E2):-
```

→	→	→	· append(L1, L2, L),
→	→	→	· L1 \= [], L2 \= [],
→	→	→	· expressio(L1, E1),
→	→	→	· expressio(L2, E2).

B

Adapta l'esquema Prolog de sota per resoldre els problemes B1, B2 i B3:

- com es poden representar els estats?
- quins passos hi ha entre estats?
- quin és el cost de cada pas?

```
main :- EstatInicial = ..., ..., EstatFinal = ...,
    between(1, 1000, CostMax), ..., % Busquem solució de cost 0; si no, de 1, etc.
    cami(CostMax, EstatInicial, EstatFinal, [EstatInicial], Cami),
    reverse(Cami, Cami1), write(Cami1), write(' amb cost '), write(CostMax), nl, halt.

cam(0, E, E, C, C). ..., % Cas base: quan l'estat actual és l'estat final
cam(CostMax, EstatActual, EstatFinal, CamiFinsAra, CamiTotal) :-
    CostMax > 0,
    unPas(CostPas, EstatActual, EstatSeguent), ..., % En B.1 i B.2, CostPas és 1
    \+ member(EstatSeguent, CamiFinsAra),
    CostMax1 is CostMax - CostPas,
    cam(CostMax1, EstatSeguent, EstatFinal, [EstatSeguent|CamiFinsAra], CamiTotal).

unPas(...) :- ...
...
```

Exemple: trobar un camí en un graf (dfs)

```
1  % Given a graph G in predicate graph(G),
2  % a path from Start to End in G is a list of vertices
3
4  graph([ [1,2], [2,3], [1,4], [4,5], [1,6], [6,7] ]).
5
6  ∨ unPas(1,A,B) :-
7      graph(E),
8      member([A,B],E).
9
10 ∨ unPas(1,A,B) :-
11     graph(E),
12     member([B,A],E).
```

```
?-main(3,7).
[3,2,1,6,7] amb cost 4
```


B1: Missioners

Busquem la manera més ràpida per tal que **3 missioners** i **3 caníbals** travessin un riu en una canoa que pot ser utilitzada per 1 o 2 persones (missioners o caníbals), però sempre evitant (per raons òbvies) que els missioners quedin en minoria en qualsevol riba.

Evidentment, no és estar en minoria:

- estar en **situació d'empat**
- que hi hagi **0 missioners** i **> 0 caníbals**

B2: Cavall

Donats

- un natural $n > 0$,
- una posició inicial I i una de final F en un tauler $n \times n$
(és a dir, $I, F \in \{1, \dots, n\} \times \{1, \dots, n\}$)
- un nombre de passos P

troba un camí de I a F en P salts del cavall d'escacs sense repetir posicions.
En cas que no hi hagi camí, el programa ha de fallar.

B3: Travessa nocturna

Quatre persones P_1 , P_2 , P_5 i P_8 volen travessar un pont de nit amb les restriccions següents:

- 1 El pont només suporta el pes de dues persones.
- 2 Hi ha una única llanterna que han de portar en travessar el pont.
- 3 Cada persona P_i triga i minuts a creuar. Dues juntes triguen com la més lenta de les dues.

C: TSP

El fitxer `tsp.pl` conté una solució a un problema semblant al **Traveling Salesman Problem**. Intenta millorar-lo tal com s'indica en el fitxer.

C: TSP

Trobar la millor ruta per visitar les primeres N ciutats començant per la 1.
Millorar el programa implementant

- 1 lowerBoundOfRemainingCities en la clàusula alternativa 2
- 2 mySelect en la clàusula 3

```
% tsp( Cities, AccumulatedKm, RouteSoFar )
%1
tsp( [], AccumulatedKm, RouteSoFar ):- storeRouteIfBetter(AccumulatedKm,RouteSoFar), fail.

%2
tsp( _, AccumulatedKm, _ ):- bestRouteSoFar(Km,_), AccumulatedKm >= Km, !, fail.
%alternative:
%tsp( Cities, AccumulatedKm, _ ):- bestRouteSoFar(Km,_),
%   lowerBoundOfRemainingCities(Cities, LBound ),    %implement this (efficiently)!
%   AccumulatedKm+LBound >= Km, !, fail.

%3
tsp( Cities, AccumulatedKm, [ CurrentCity | RouteSoFar ] ):-
    select( City, Cities, RemainingCities ), % select next city to visit
%   myselect( CurrentCity, City, Cities, RemainingCities ), % implement this
    distance( CurrentCity, City, Km ), AccumulatedKm1 is AccumulatedKm+Km,
    tsp( RemainingCities, AccumulatedKm1, [ City, CurrentCity | RouteSoFar ] ).
```